

SCC-0570 - Capítulo 5
Perceptron Multicamadas

João Luís Garcia Rosa[1]

[1]Instituto de Ciências Matemáticas e de Computação
Universidade de São Paulo - São Carlos
http://www.icmc.usp.br/~joaoluis

2019

Sumário

1. Introdução
   - O Perceptron de camada única
   - O Perceptron Multicamadas
2. *Back-propagation* (BP)
   - Algoritmo LMS
3. MLPs
   - XOR
   - Generalização
   - Aproximação de Funções
4. *Deep learning*
   - Redes Convolucionais

Sumário

1. Introdução
   - O Perceptron de camada única
   - O Perceptron Multicamadas
2. *Back-propagation* (BP)
   - Algoritmo LMS
3. MLPs
   - XOR
   - Generalização
   - Aproximação de Funções
4. *Deep learning*
   - Redes Convolucionais

O perceptron

- Como já visto, o primeiro modelo matemático do neurônio foi proposto por McCulloch & Pitts em 1943 [3].
- Mais tarde em 1957, Rosenblatt [5] criou o modelo do perceptron.
- Um perceptron modela um neurônio tomando uma soma ponderada de suas entradas e enviando a saída 1 (*spike*) se esta soma é maior que um determinado limiar de ativação.
- O perceptron, com função de ativação linear, pode ser modelado como um discriminador linear:
  - dados 2 pontos, uma reta é capaz de discriminar esses 2 pontos,
  - para algumas configurações de m pontos, uma reta é capaz de separar estes pontos em 2 classes.

Limitações do perceptron de camada única

- O perceptron é uma rede *feedforward* (não recorrente) de uma única camada.
- O perceptron só é capaz de aprender a solução de problemas linearmente separáveis.
- O algoritmo de aprendizagem do perceptron (regra delta) não funciona com redes de mais de uma camada.

Sumário

1. Introdução
   - O Perceptron de camada única
   - O Perceptron Multicamadas
2. *Back-propagation* (BP)
   - Algoritmo LMS
3. MLPs
   - XOR
   - Generalização
   - Aproximação de Funções
4. *Deep learning*
   - Redes Convolucionais

Mais camadas

- Redes Neurais alimentadas à frente com múltiplas camadas:
  - Um conjunto de unidades sensoriais (nós fonte): *camada de entrada*.
  - Uma ou mais *camadas escondidas* de nós computacionais.
  - Uma *camada de saída* de nós computacionais.
  - O sinal de entrada se propaga através da rede numa direção às frente, camada por camada.
  - Essas redes são conhecidas como perceptrons multicamadas (MLP), uma generalização do perceptron de camada única.

Perceptrons Multicamadas

- MLP têm sido aplicados com sucesso para resolver diversos e difíceis problemas, incluindo aplicações de **deep learning**.
- Treinados de forma supervisionada
- Algoritmo muito popular: *error back-propagation* [1, 6], baseado na regra de aprendizado de correção de erro:
  - Generalização do algoritmo do filtro adaptativo: LMS.
  - Duas fases: um passo à frente e um passo para trás.
  - No passo à frente, um padrão de atividade (vetor de entrada) é aplicado aos nós sensoriais da rede e seu efeito é propagado através da rede camada por camada.
  - Por fim, um conjunto de saídas é produzido como resposta real da rede.

Perceptrons Multicamadas

- BP:
  - Durante o passo à frente, os pesos sinápticos são *fixos*.
  - Durante o passo para trás, os pesos sinápticos são *ajustados* de acordo com uma regra de correção de erro.
  - Especificamente, a resposta real é subtraída da resposta desejada para produzir um *sinal de erro*.
  - Este sinal de erro é propagado de volta através da rede, em sentido contrário aos estímulos.
  - Os pesos são ajustados para fazer a resposta real se aproximar da resposta desejada, de uma forma estatística.

Características do MLP

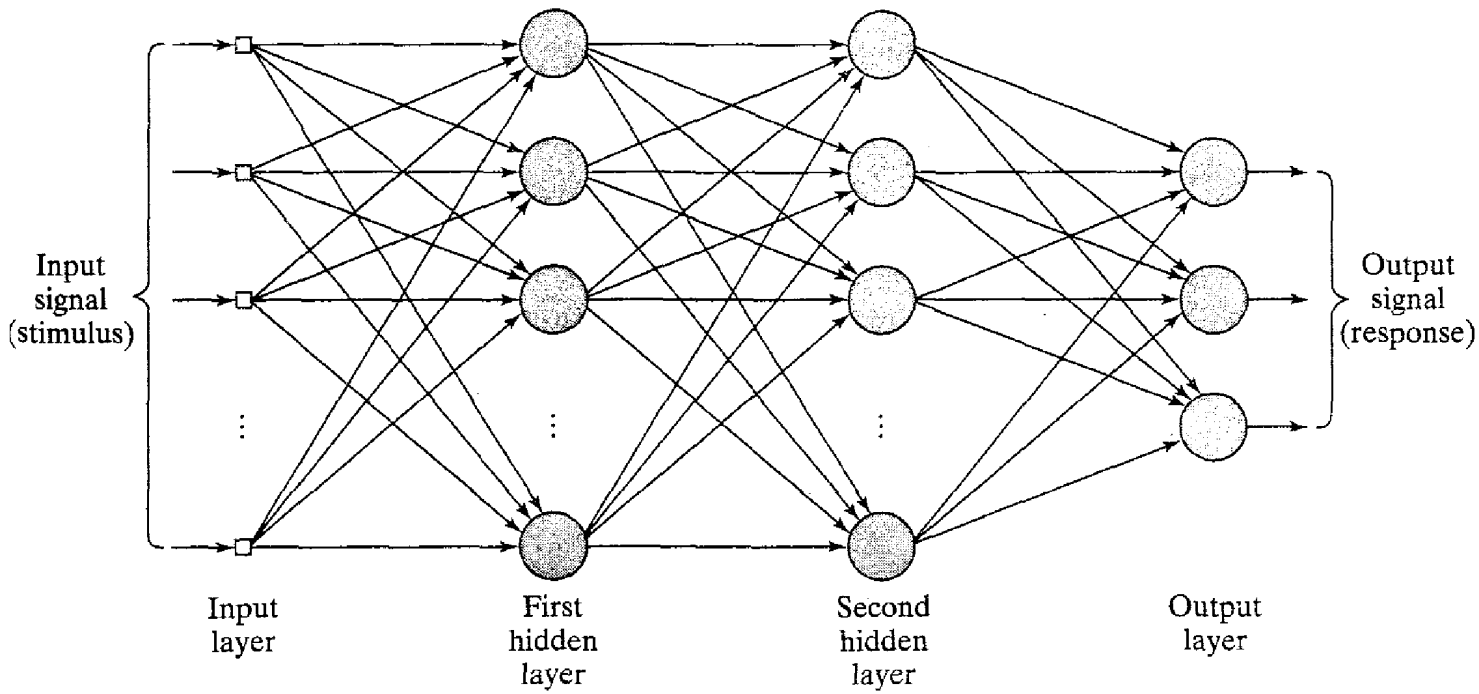
- Características do MLP:
  1. O modelo de cada neurônio da rede inclui uma *função de ativação não-linear*, a não-linearidade sigmoidal definida pela função logística:

$$y_j = \frac{1}{1 + exp(-v_j)} \tag{1}$$

     onde v_j é o campo local induzido (isto é, a soma ponderada das entradas mais o *bias*) do neurônio j e y_j é a saída.
  2. A rede contém uma ou mais camadas de *neurônios escondidos*, que não são parte da entrada nem da saída.
  3. A rede exibe um alto grau de *conectividade*, determinado pelas sinapses.

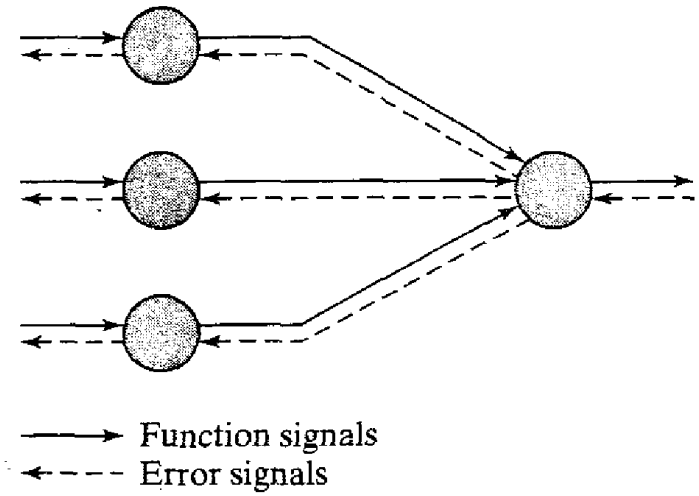
MLP

- A figura abaixo [2] mostra um grafo arquitetural de um perceptron multi-camadas com duas camadas escondidas e uma camada de saída.
- Esta rede é *totalmente conectada*.



MLP

- A figura abaixo [2] mostra uma porção de um MLP.
- Dois tipos de sinais são identificados:
  1. *Sinais de função*: sinal de entrada (estímulo) que chega na entrada, propaga-se para frente (neurônio por neurônio) através da rede e emerge na saída da rede como um sinal de saída. Também conhecido como *sinais de entrada*.
  2. *Sinais de erro*: origina em um neurônio de saída da rede e propaga-se de volta (camada por camada) através da rede.



MLP

- Cada neurônio escondido ou de saída de um MLP realiza duas computações:
  1. Sinal de função que aparece na saída de um neurônio, expresso como um função não-linear contínua do sinal de entrada e pesos associados.
  2. Estimativa do vetor gradiente (isto é, os gradientes da superfície de erro com respeito aos pesos conectados às entradas de um neurônio), necessária para o passo para trás através da rede.

Sumário

1. Introdução
   - O Perceptron de camada única
   - O Perceptron Multicamadas
2. *Back-propagation* (BP)
   - Algoritmo LMS
3. MLPs
   - XOR
   - Generalização
   - Aproximação de Funções
4. *Deep learning*
   - Redes Convolucionais

Algoritmo de aprendizagem LMS

- Algoritmo de aprendizagem usado por perceptrons de uma única camada.
- LMS = *Least-Mean-Square*.
- Seja um conjunto de treinamento composto de l padrões de entrada/saída desejada.
- Como a rede tem m unidades de entrada x e n unidades de saída desejada d, cada um dos l padrões é do tipo:

$$((x_1, ..., x_m), (d_1, ..., d_n))$$

- Uma vez que se apresenta à rede os l padrões de treinamento, pode-se obter uma medida do erro produzido pela rede.
- O erro é função:
  - de cada padrão, e
  - do erro produzido em cada unidade de saída, quando cada padrão é apresentado.

Treinamento Supervisionado: Regra delta

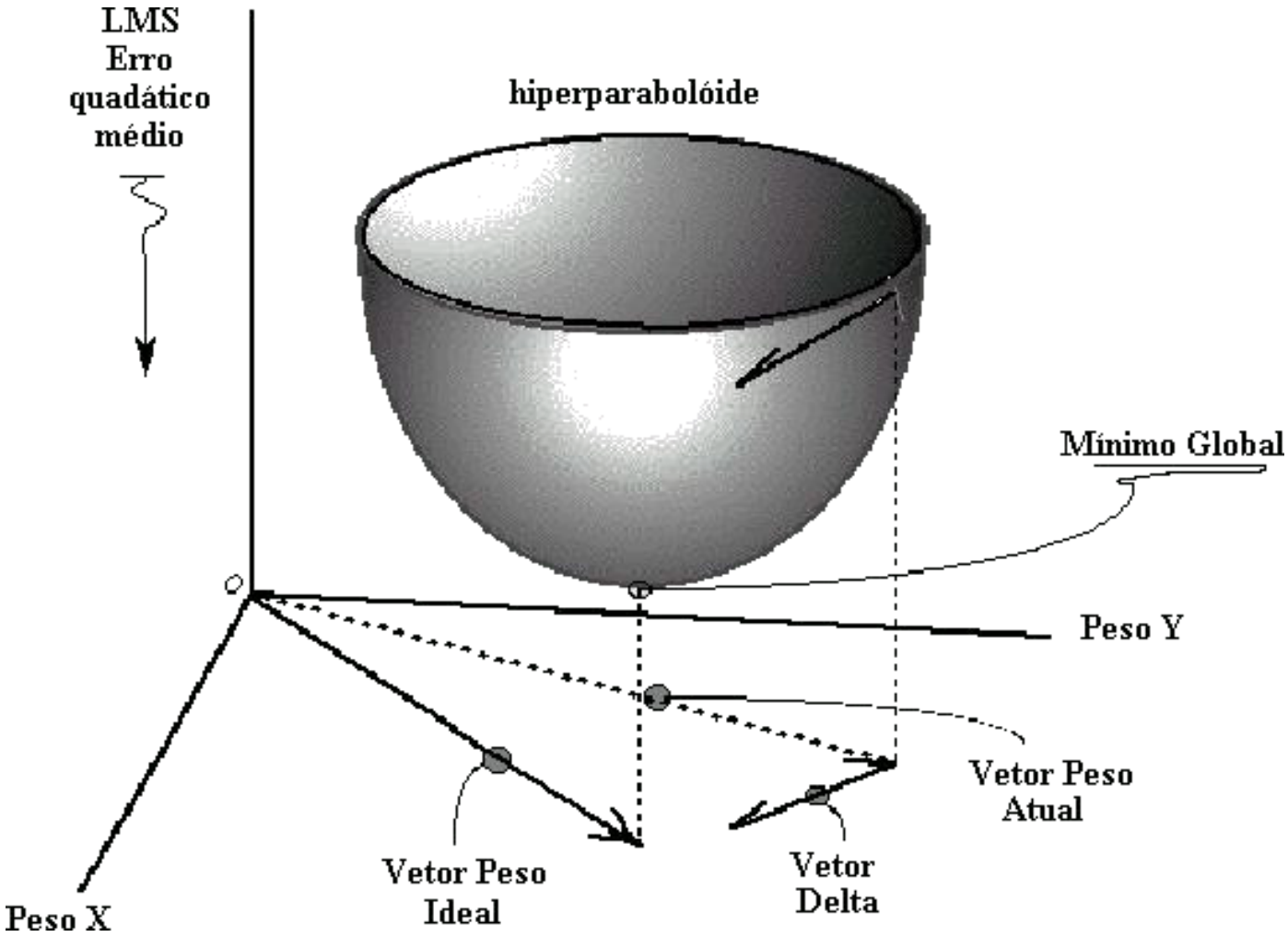


Figure 1: A direção do gradiente negativo é a de descida mais íngreme (*steepest descent*).

Algoritmo de aprendizagem LMS

- Se...
  - a rede aprende perfeitamente os padrões de treinamento, e
  - os padrões de treinamento refletem perfeitamente a tarefa que se quer aprender
- Então...
  - após o treinamento, o erro será zero.

- A principal causa do erro vem das diferenças entre saída real e saída desejada, que decorre da saída produzida por pesos (e *biases*) incorretos.
- Aprender significa achar os pesos que tornem mínimo o erro.

Algoritmo de aprendizagem LMS

- O erro total ξ após o treinamento é:

$$\xi(w) = \sum_{p=1}^{l} \xi_p \tag{2}$$

onde ξ_p é o erro produzido quando o p-ésimo padrão de treinamento é apresentado à rede.

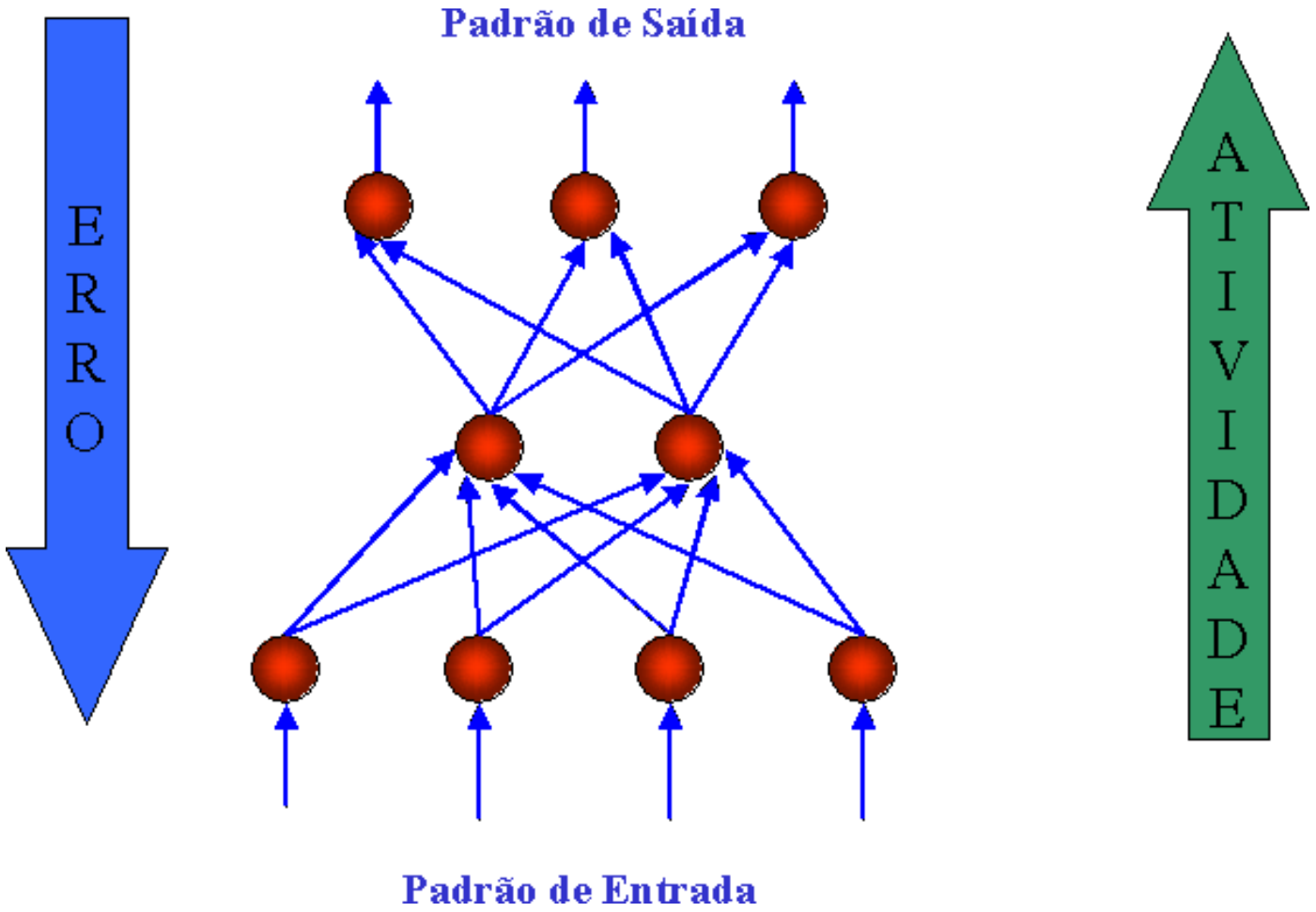
- O erro ξ_p pode ser medido de várias maneiras, mas a medida mais usada é o *erro quadrático médio*:

$$\xi_p = \frac{1}{2} \sum_{k=1}^{n} (d_k - y_k)^2 \tag{3}$$

Back-propagation

- A dificuldade de aplicar a regra delta: como calcular o erro para uma unidade da camada escondida?
- Solução: (*Error*) *Back-propagation* $\Rightarrow$ Generalização da regra delta.
- *Back-propagation* é um algoritmo supervisionado, que requer duas fases:
  - propagação da ativação, e
  - retropropagação do erro.
- permite calcular o erro baseado só em informação local: disponível nos pesos e unidades próximas ao peso que está sendo modificado (como no SNC).

Error Back-propagation



Back-propagation

- Os problemas do *back-propagation*:
  - É bastante caro computacionalmente (lento),
  - Não resolve bem problemas de grande porte,
  - Às vezes, a solução encontrada é um mínimo local - um valor localmente mínimo para a função erro.
- Vantagens do *back-propagation*:
  - poder de aproximação universal:
    - dada uma função contínua, existe uma rede de duas camadas (uma escondida) que pode ser treinada por *back-propagation* de modo a aproximar o quanto se queira essa função.
  - algoritmo mais usado.

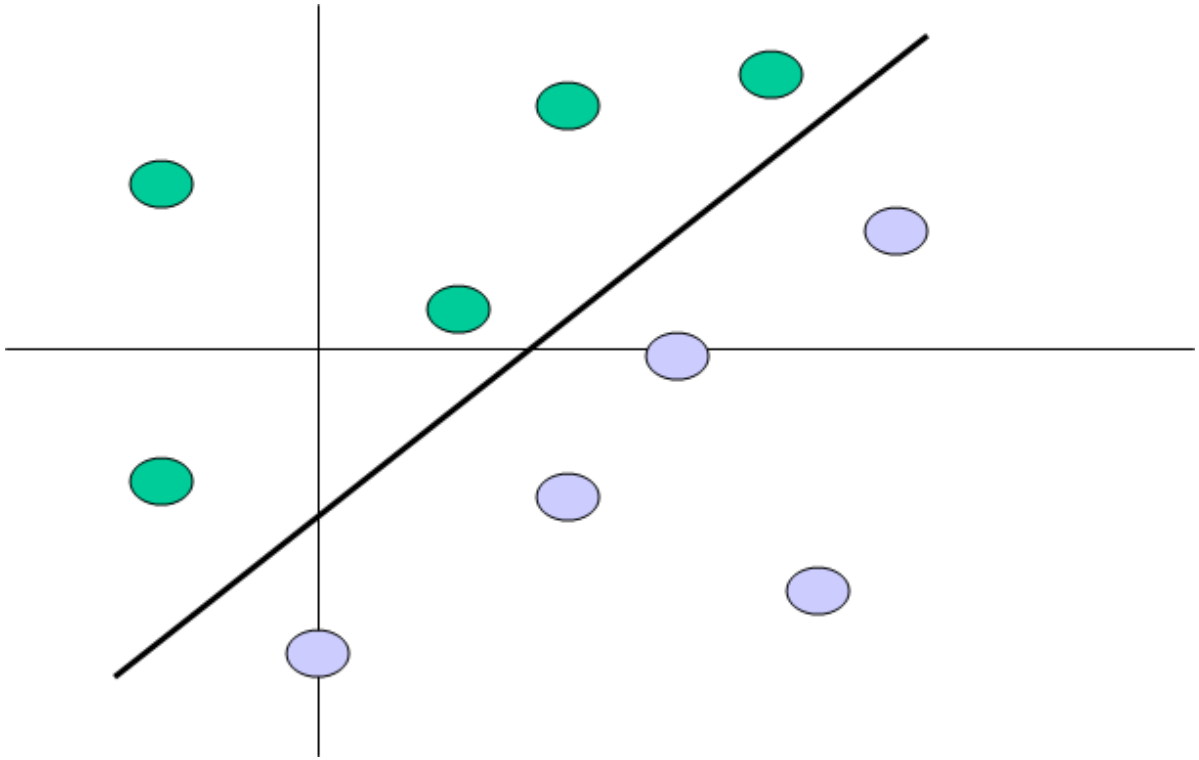
Sumário

1. Introdução
   - O Perceptron de camada única
   - O Perceptron Multicamadas
2. *Back-propagation* (BP)
   - Algoritmo LMS
3. MLPs
   - XOR
   - Generalização
   - Aproximação de Funções
4. *Deep learning*
   - Redes Convolucionais

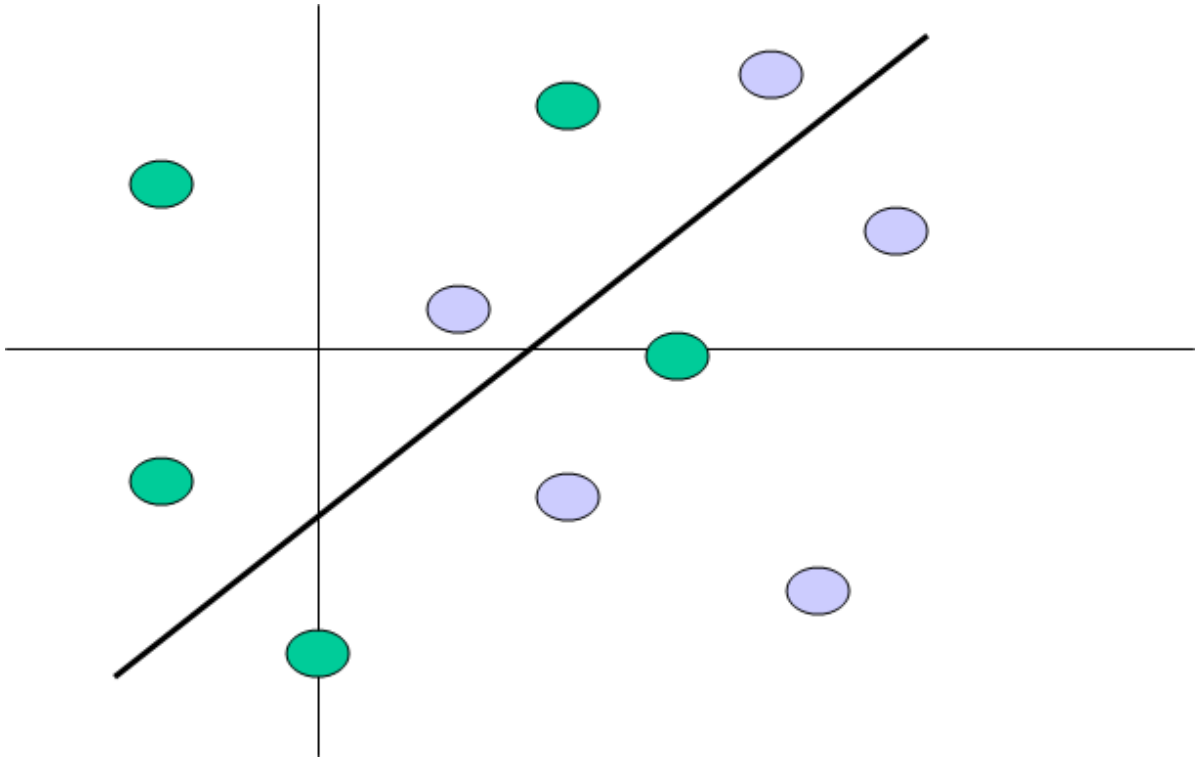
Problema XOR

- No perceptron de única camada não há neurônio escondido, porisso não é possível classificar padrões de entrada que não são linearmente separáveis.
- Entretanto, padrões não linearmente separáveis são comuns.
- Por exemplo, o problema do ou-exclusivo (XOR), que pode ser visto como um caso especial de um problema mais geral, a classificação de pontos no *hipercubo unitário*.
- Cada ponto no hipercubo ou é da classe 0 ou da classe 1.
- Porém, no caso do XOR, precisa-se considerar apenas os quatro cantos do *quadrado unitário*, que correspondem aos padrões de entrada (0,0), (0,1), (1,1) e (1,0).

Conjunto de pontos linearmente separáveis

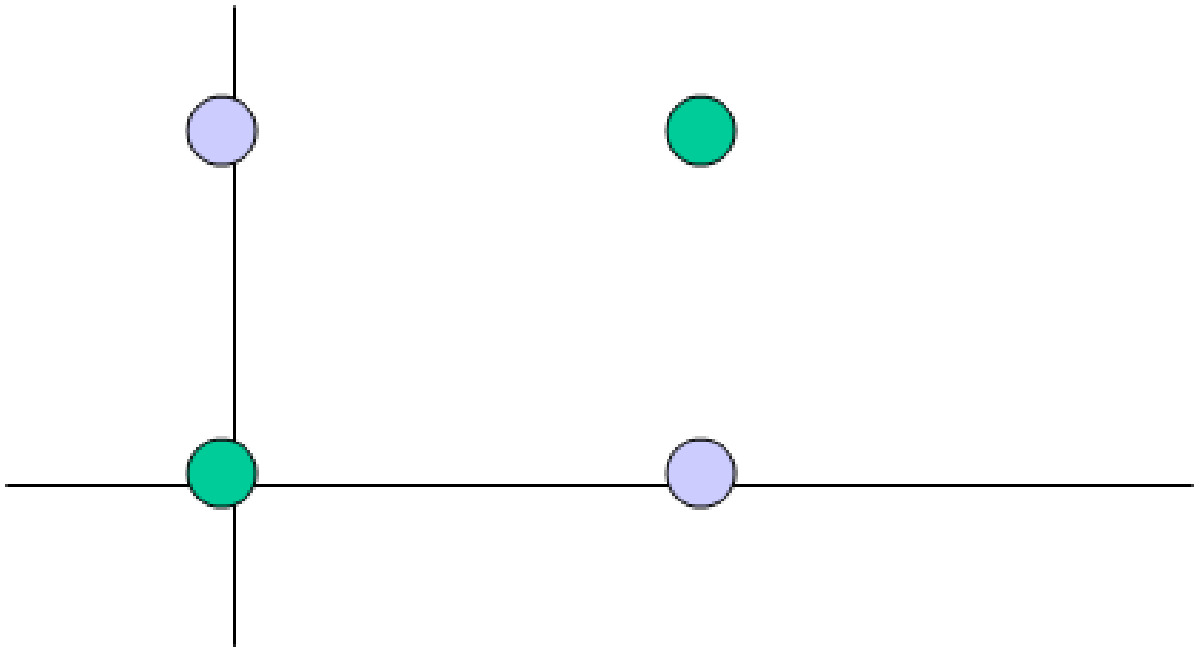


Conjunto de pontos *não*-linearmente separáveis (por 1 reta)



OU Exclusivo

- $\{(0,0),0;(0,1),1;(1,0),1;(1,1),0\}$



- E por n retas?

OU Exclusivo

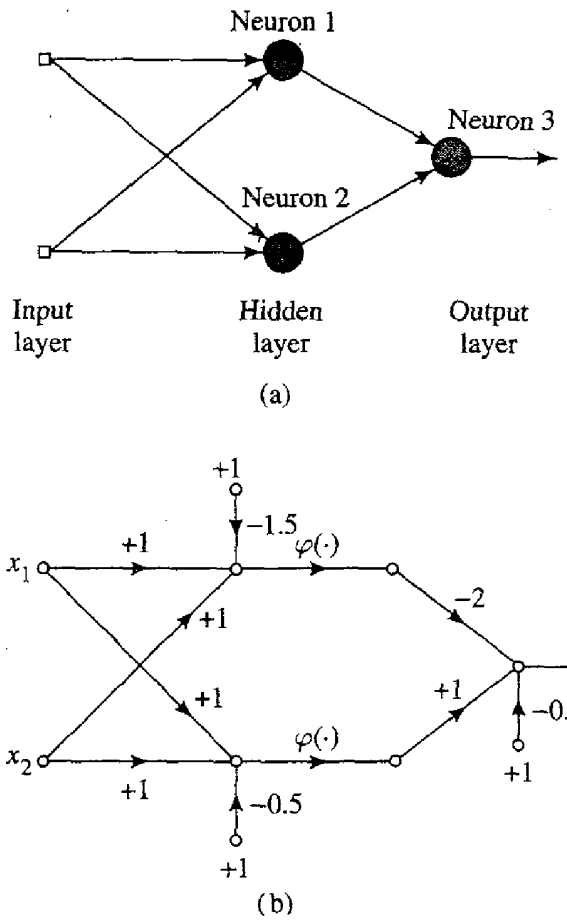
- Pode-se resolver o problema XOR usando uma única camada escondida com dois neurônios (veja figura 2*a*).
- O grafo do fluxo de sinal é mostrado na figura 2*b*. Assume-se:
  - Cada neurônio é representado por um modelo de McCulloch-Pitts [3], que usa uma função limiar como função de ativação.
  - Os bits 0 e 1 são representados pelos níveis 0 e +1, respectivamente.
- O neurônio 1 da camada escondida é caracterizado como:

$$w_{11} = w_{12} = +1 \qquad b_1 = -\frac{3}{2} \tag{4}$$

- A inclinação do limite de decisão construído por esse neurônio escondido é igual a -1 (veja figura 3*a* [2]).

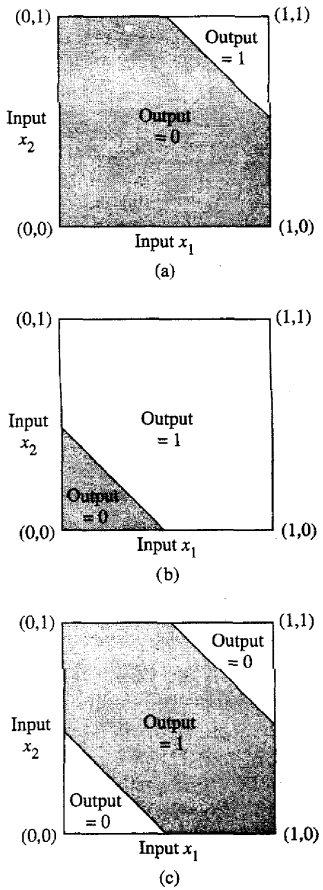
OU Exclusivo

Figure 2: (a) Grafo arquitetural da rede para resolver o problema do XOR. (b) Grafo de fluxo de sinais da rede [2].



OU Exclusivo

Figure 3: (a) Fronteira de decisão construída pelo neurônio escondido 1 da rede da figura 2. (b) Fronteira de decisão construída pelo neurônio escondido 2 da rede. (c) Fronteiras de decisão construídas pela rede completa [2].



OU Exclusivo

- O neurônio 2 da camada escondida é caracterizado por:

$$w_{21} = w_{22} = +1 \qquad b_2 = -\frac{1}{2} \tag{5}$$

- A orientação e posição do limite de decisão construído pelo segundo neurônio são mostrados na figura $3b$ [2].
- O neurônio de saída 3 é caracterizado como:

$$w_{31} = -2 \qquad w_{32} = +1 \qquad b_3 = -\frac{1}{2} \tag{6}$$

- A função desse neurônio de saída é construir uma combinação linear dos limites de decisão formados pelos dois neurônios escondidos (figura $3c$ [2]).

Sumário

1. Introdução
   - O Perceptron de camada única
   - O Perceptron Multicamadas
2. *Back-propagation* (BP)
   - Algoritmo LMS
3. MLPs
   - XOR
   - Generalização
   - Aproximação de Funções
4. *Deep learning*
   - Redes Convolucionais

Generalização

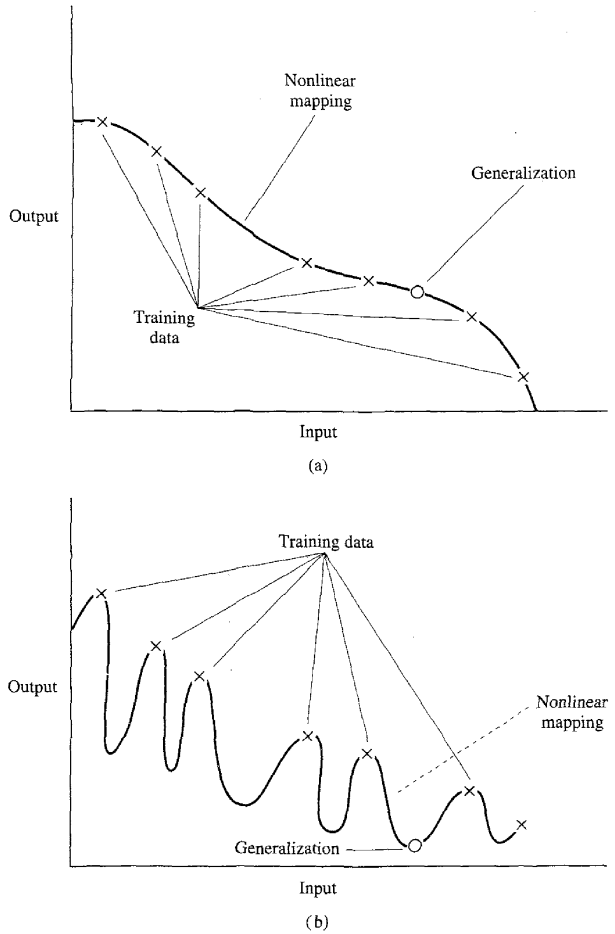
- No aprendizado BP, começa-se com uma amostra de treinamento e usa-se o algoritmo BP para computar os pesos sinápticos de um MLP carregando (codificando) o máximo possível de exemplos de treinamento na rede.
- A esperança é que a RNA projetada *generalize*: quando os mapeamentos entrada-saída computados pela rede são corretos (ou próximos) para dados de teste **nunca** usados na criação ou no treinamento da rede.
- Assume-se que os dados de teste são retirados da mesma população usada para gerar o conjunto de treinamento.
- O processo de aprendizado (treinamento de uma rede neural) pode ser visto como um problema de "preenchimento de curva."

Generalização

- Veja na figura 4*a* como a generalização pode ocorrer em uma rede hipotética.
- O mapeamento de entrada-saída não linear representado pela curva é computado pela rede como um resultado do aprendizado dos pontos "training data".
- O ponto "generalization" é o resultado da interpolação feita pela rede.
- Problema: quando a rede aprende muitos exemplos de entrada-saída pode terminar memorizando os dados de treinamento.
- Isso pode fazer com que a RNA ache uma característica (devido ao ruído) que está presente nos dados de treinamento, mas que não é verdadeira para a função modelada: overfitting ou overtraining.
- Quando uma rede é "overtrained", perde a habilidade de generalizar.

Generalização

Figure 4: (a) Dados ajustados apropriadamente (boa generalização). (b) Dados *overfitted* (generalização pobre) [2].



Generalização

- Um exemplo de como a generalização pobre devido à memorização em uma RNA pode ocorrer é ilustrada na figura 4*b* [2], para os mesmos dados da figura 4*a*.
- "Memorização" é essencialmente uma "look-up table", que implica que o mapeamento de entrada-saída computado pela rede não é suave.
- A suavidade do mapeamento entrada-saída é relacionada aos critérios de seleção de modelo como a *navalha de Occam*[1]: selecionar a função "mais simples" na ausência de qualquer conhecimento prévio.
- Aqui, a função mais simples significa a função mais suave que aproxima o mapeamento para um dado critério de erro, porque isso acarreta menos recursos computacionais.

[1] Lei da Economia: a explicação mais simples é a correta.

Sumário

1. Introdução
   - O Perceptron de camada única
   - O Perceptron Multicamadas
2. *Back-propagation* (BP)
   - Algoritmo LMS
3. MLPs
   - XOR
   - Generalização
   - Aproximação de Funções
4. *Deep learning*
   - Redes Convolucionais

Aproximações de funções

- Um MLP treinado com BP pode realizar um *mapeamento de entrada-saída não-linear*.
- Seja m_0 o número de nós de entrada de um MLP e $M = m_L$ o número de neurônios na camada de saída.
- O relacionamento entrada-saída da rede define um mapeamento de um espaço de entrada Euclidiano m_0-dimensional para um espaço de saída Euclidiano M-dimensional, que é infinitamente continuamente diferenciável quando a função de ativação também é.
- Ao avaliar a capacidade de um MLP a partir desse ponto de vista de mapeamento entrada-saída, surge a seguinte questão fundamental:

  Qual é o número mínimo de camadas escondidas em um MLP com um mapeamento entrada-saída que provê uma realização aproximada de qualquer mapeamento contínuo?

Teorema da Aproximação Universal

- A resposta a essa pergunta está no Teorema da Aproximação Universal (TAU).
- O TAU é diretamente aplicável ao MLP.
- O TAU é um *teorema de existência*, já que provê a justificativa matemática para a aproximação de uma função contínua arbitrária.
- Estabelece que *uma única camada escondida é suficiente para um MLP computar uma aproximação* ϵ *uniforme para um dado conjunto de treinamento representado pelo conjunto de entradas* $x_1, ..., x_{m_0}$ *e uma saída desejada* $f(x_1, ..., x_{m_0})$.
- No entanto, o teorema não diz que uma única camada escondida é ótima no sentido de tempo de aprendizado, facilidade de implementação ou (mais importante) generalização: **deep learning**.

Sumário

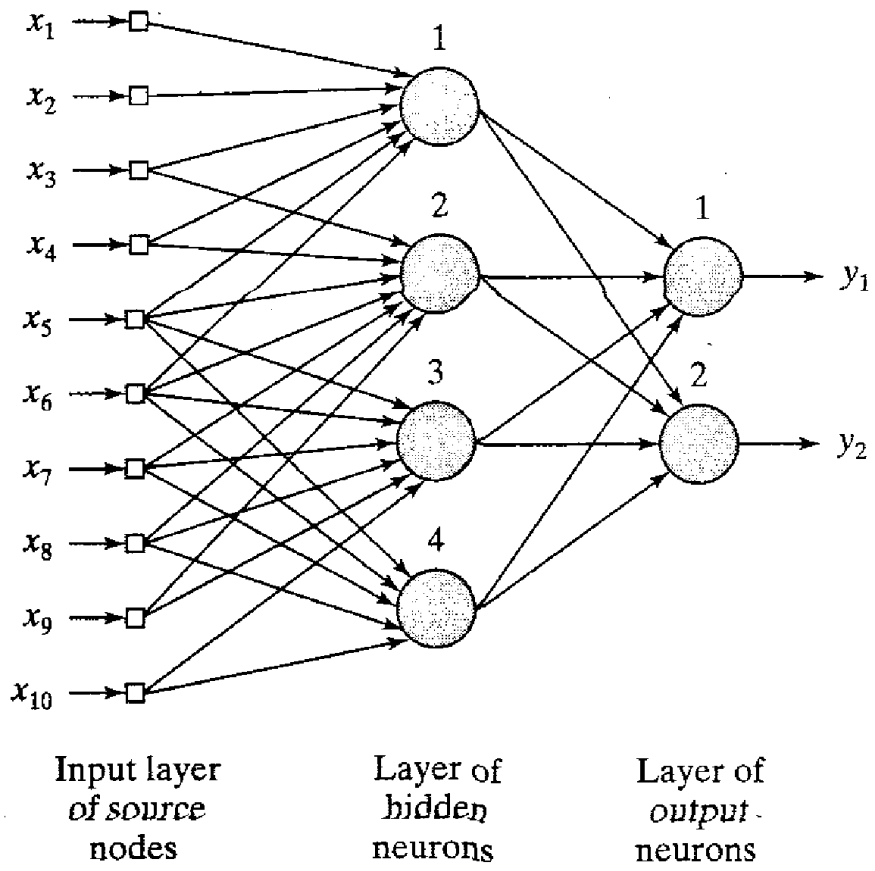
1. Introdução
   - O Perceptron de camada única
   - O Perceptron Multicamadas
2. *Back-propagation* (BP)
   - Algoritmo LMS
3. MLPs
   - XOR
   - Generalização
   - Aproximação de Funções
4. *Deep learning*
   - Redes Convolucionais

Como incluir informação prévia no projeto de uma RNA

- Procedimento *ad-hoc* para construir informação prévia no *design* de uma RNA:
  1. *Restringir a arquitetura da rede* através do uso de conexões locais conhecidas como *campos receptivos*.
  2. *Limitar a escolha dos pesos sinápticos* através do uso do *compartilhamento de pesos*.
- Essas duas técnicas, especialmente a segunda, têm um efeito colateral benéfico: o número de parâmetros livres na rede é reduzido significativamente.
- Considere a rede *feedforward* parcialmente conectada da figura 5.
- Essa rede apresenta uma arquitetura restringida por construção.
- Os seis nós fonte superiores constituem o campo receptivo para o neurônio escondido 1.

Como incluir informação prévia no projeto de uma RNA

Figure 5: Ilustração do uso combinado do campo receptivo e compartilhamento de pesos [2].



Como incluir informação prévia no projeto de uma RNA

- Para satisfazer a restrição do compartilhamento de pesos, deve-se usar o mesmo conjunto de pesos sinápticos para cada um dos neurônios da camada escondida.
- Portanto, para seis conexões locais por neurônio escondido e um total de quatro neurônios escondidos (figura 5), pode-se expressar o campo local induzido do neurônio escondido j como (**soma de convolução**):

$$v_j = \sum_{i=1}^{6} w_i x_{i+j-1}, \qquad j = 1, 2, 3, 4 \tag{7}$$

onde $\{w_i\}_{i=1}^{6}$ constitui o mesmo conjunto de pesos compartilhados por todos os quatro neurônios escondidos e x_k é o sinal do nó fonte $k = i + j - 1$.

- É por essa razão que uma rede *feedforward* usando conexões locais e compartilhamento de pesos na forma descrita é conhecida como **rede convolucional**.

Convolução

- Foco no *layout* estrutural do MLP: classe especial chamada redes convolucionais.
- Uma rede convolucional é um MLP projetado para reconhecer formas bi-dimensionais com um alto grau de invariância para translação, mudança de escala, e outras formas de distorção.
- Esta tarefa difícil é aprendida de uma maneira supervisionada por uma rede cuja estrutura inclui as seguintes formas de *restrições*:
  1. *Extração de características*,
  2. *Mapeamento de características*,
  3. *Sub-amostragem*.

Convolução

- Restrições:
  1. *Extração de características*: Cada neurônio obtém suas entradas sinápticas de um *campo receptivo* local da camada anterior, forçando-o a extrair características locais. A posição relativa de uma característica extraída em relação às outras é preservada.
  2. *Mapeamento de características*: Cada camada computacional da rede é composta de múltiplos *mapas de características*, onde cada mapa tem a forma de um plano no qual os neurônios individuais são *restringidos* para compartilhar o mesmo conjunto de pesos sinápticos. Efeitos benéficos:
     - *Invariância de deslocamento*: uso da *convolução* seguido de uma função sigmoide (achatamento).
     - *Redução no número de parâmetros livres*, acompanhado de *compartilhamento de pesos*.

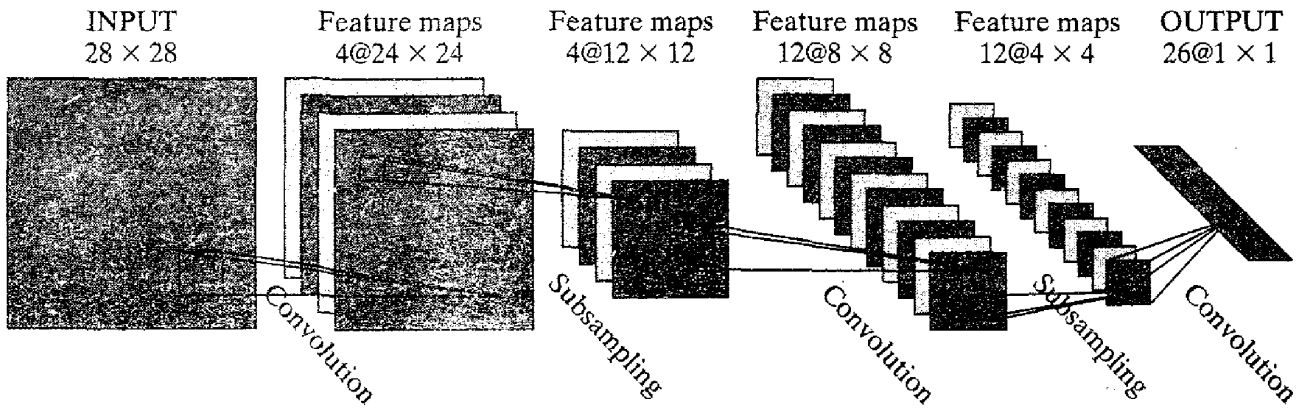
Convolução

- Restrições (cont.):
  3. *Sub-amostragem*: cada camada convolucional é seguida por uma camada computacional que realiza *cálculo da média local* e *sub-amostragem*, onde a resolução do mapa de características é reduzida. Tem o efeito de reduzir a sensibilidade da saída do mapa a deslocamentos e outras formas de distorção.
- Todos os pesos em todas as camadas de uma rede convolucional são aprendidos através do treinamento.
- No entanto, a rede aprende a extrair suas próprias características automaticamente.

Convolução

- A figura 6 mostra um *layout* arquitetural de uma rede convolucional com uma camada de entrada, quatro camadas escondidas e uma camada de saída.
- Esta rede foi projetada para realizar *processamento de imagem*: reconhecimento de caracteres manuscritos.

Figure 6: Rede convolucional para processamento de imagens tal como reconhecimento de escrita manual [2].



Convolução

- A camada de entrada, com 28 $\times$ 28 nós sensoriais, recebe as imagens de diferentes caracteres, centralizados e normalizados.
- Então, a computação alterna entre convolução e sub-amostragem:
  - 1ª escondida: convolução. 4 mapas de características com cada mapa consistindo de 24 $\times$ 24 neurônios. Cada neurônio tem um campo receptivo de tamanho 5 $\times$ 5.
  - 2ª escondida: sub-amostragem e média local. 4 mapas de características com cada mapa consistindo de 12 $\times$ 12 neurônios. Cada neurônio tem um campo receptivo de tamanho 2 $\times$ 2, um coeficiente treinável, um *bias* treinável e uma função de ativação sigmoide.
  - 3ª escondida: convolução. 12 mapas de características com cada mapa consistindo de 8 $\times$ 8 neurônios. Cada neurônio tem conexões sinápticas com vários mapas de características das camadas escondidas anteriores.

Convolução

- Computação (cont.):
  - 4ª escondida: sub-amostragem e média local. 12 mapas de características com cada mapa consistindo de 4×4 neurônios.
  - A camada de saída realiza um estágio final da convolução. Consiste de 26 neurônios, atribuídos a um dos 26 caracteres possíveis. Cada neurônio tem um campo receptivo de tamanho 4×4.
- Efeito "bipiramidal:" a cada camada convolucional ou de sub-amostragem, o número de mapas de características é aumentado enquanto que a resolução espacial é reduzida, comparada a camada anterior correspondente.
- O MLP da figura 6 contém 100.000 conexões sinápticas mas apenas 2.600 parâmetros livres.
- Esta redução dramática é conseguida através do uso de compartilhamento de pesos.

Convolução

- A capacidade da máquina de aprendizado é reduzida, que por sua vez, aumenta sua habilidade de generalização.
- Os ajustes dos parâmetros livres são feitos usando uma forma estocástica (sequencial) do aprendizado *back-propagation*.
- O uso de compartilhamento de pesos torna possível implementar a rede convolucional de forma paralela: outra vantagem sobre o MLP totalmente conectado.
- Duas lições (figura 6):
  1. Um MLP de tamanho gerenciável é capaz de aprender um mapeamento complexo, de alta dimensão e não-linear *restringindo* seu projeto através da incorporação de conhecimento prévio sobre a tarefa.
  2. Os níveis dos pesos sinápticos e *bias* podem ser aprendidos repetindo o algoritmo BP simples através do conjunto de treinamento.

Bibliografia I

[1] A. E. Bryson and Y.-C. Ho
Applied Optimal Control.
Blaisdell, New York, 1969.

[2] S. Haykin
Neural networks - a comprehensive foundation.
2nd. edition. Prentice Hall, 1999.

[3] W. S. McCulloch and W. Pitts
A logical calculus of the ideas immanent in nervous activity
Bulletin of Mathematical Biophysics, 5, pp. 115-133, 1943.

[4] R. A. F. Romero
SCC-5809 Redes Neurais.
Slides e listas de exercícios. Programa de Pós-Graduação em Ciência de Computação e Matemática Computacional. ICMC/USP, 2010.

Bibliografia II

[5] F. Rosenblatt
The perceptron: A perceiving and recognizing automaton.
Report 85-460-1, Project PARA, Cornell Aeronautical Lab., Ithaca, NY, 1957.

[6] D. E. Rumelhart, G. E. Hinton, and R. J. Williams
Learning representations of back-propagation errors.
Nature (London), vol. 323, pp. 533–536, 1986.